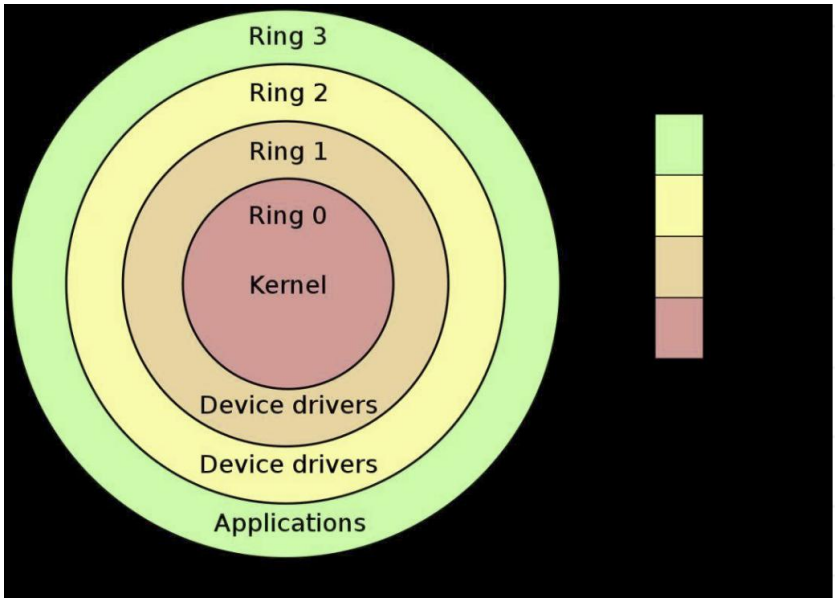
Linux 系统学习笔记

一、 基础知识

1．CPU 权限级别

虽然不同的 CPU 架构拥有不同的指令集，但是它们在设计的时候，寄存器上都会有一个两比特的模式位，有 R0 和到 R3 四种模式，数字越小，权限越高。

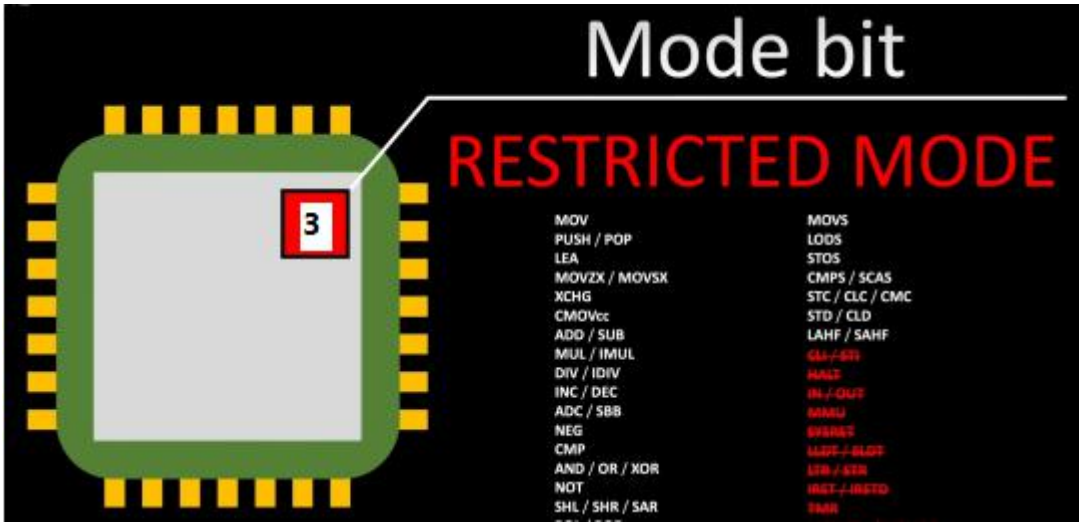


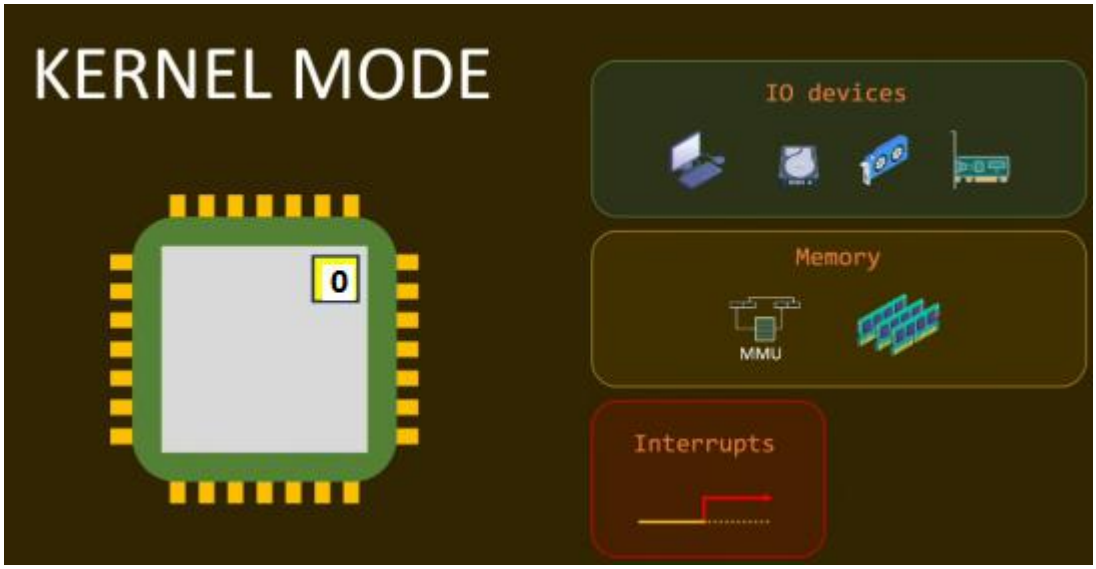
其中 R1、R2 权限级别在 CPU 设计时是给第三方厂商的驱动程序预留的，但是在现代操作系统中，R1、R2 级别几乎不被使用，驱动程序会被操作系统直接赋予 R0 级别的权限。

所以下面只讨论 R0 和 R3 级别

当模式位为 R0 级别时，允许线程执行 CPU 所有的指令

当模式位为 R3 级别时，只允许线程执行内存中数据的运算、复制等普通指令，对 IO 设备（即硬件设备：硬盘、网卡、显卡、屏幕、键盘、鼠标等）、内存管理及跳转等指令的访问将被禁止。





2．操作系统和 CPU

（1）操作系统加载顺序

（关于虚拟内存、页表的相关知识见 IO 笔记）

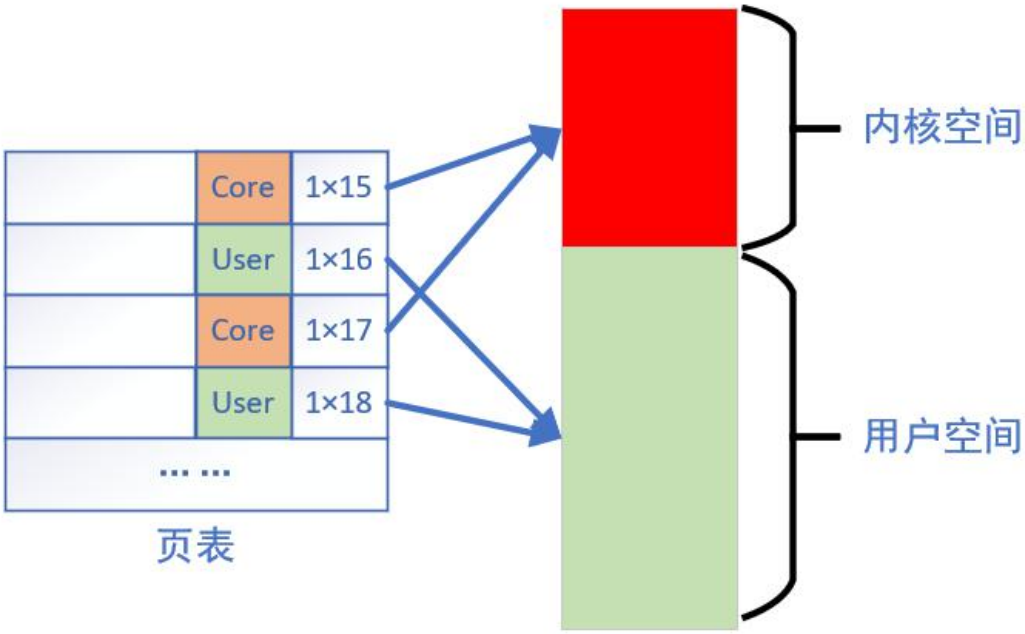
①当操作系统启动时，系统的核心（包括驱动程序）会以实模式（一种权限接近 R0 的模式）加载到内存中；

②然后会将寄存器的模式位置为 R0 权限，并初始化内存管理和页表，通过页表的标记来分配内核空间和用户空间；

③指向系统核心程序的内存空间的页表标记为内核空间；其他的空间页表标记为用户空间；

④操作系统启动，并将寄存器的模式位置为 R3；

⑤然后创建第一个用户进程（PID = 1），对用户空间进行服务；



（2）操作系统的 OS API

① 概述

在页表将内存空间分为内核空间和用户空间后

内核空间的程序拥有 R0 级别的权限，即可以执行 CPU 的全部指令集

用户空间的程序只有 R3 级别的权限，只能进行内存中数据的运算、复制等普通指令，对 IO 设备的访问会被禁止；

为了让用户的软件或程序也能访问 IO 设备，操作系统将特权指令封装成了 syscall 接口，着操作系统内核加载内核空间，可以让用户能访问硬件设备且加入了安全限制，不至于将操作系统或者硬件损坏。

EXAMPLES OF WINDOWS AND UNIX SYSTEM CALLS

The following illustrates various equivalent system calls for Windows and UNIX operating systems.

	Windows	Unix, Linux and MacOS
Process control	CreateProcess()	fork()
	ExitProcess()	exit()
	WaitForSingleObject()	wait()
File management	CreateFile()	open()
	ReadFile()	read()
	WriteFile()	write()
	CloseHandle()	close()

（3）用户程序对 OS API 的调用

用户的程序最终都会被转换为 CPU 指令集，当用户要对硬件进行访问时，如磁盘文件读取；

通过 syscall 指令调用内核空间的由操作系统提供的接口，完成对 IO 设备的调用。

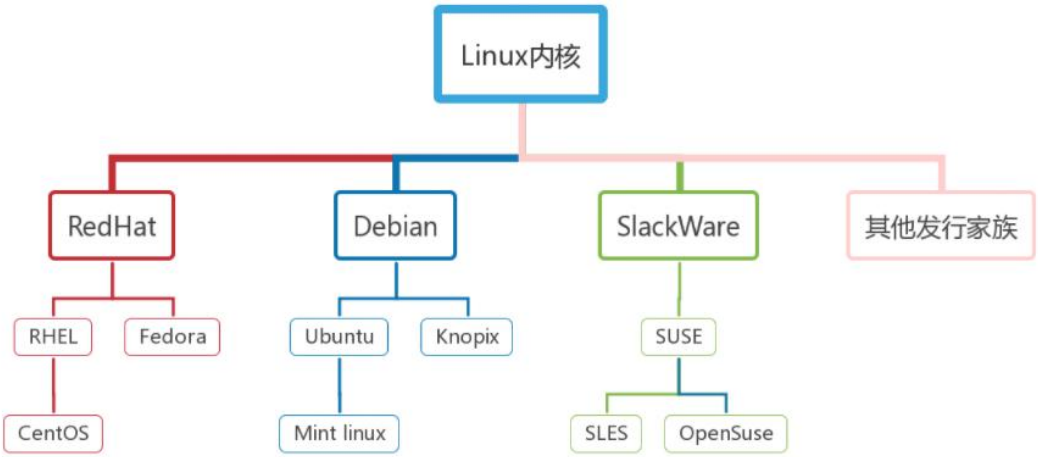
（4）内核空间的拓展

操作系统厂商并不可能对所有的硬件设备编写调用函数，比如英伟达推出了新款的显卡，他们有自己的安全访问规范，这就要求厂商自己开发调用函数，即驱动；

所以操作系统厂商推出了安全证书这一概念，无论什么软件,只要通过操作系统的驱动加载机制并获取到操作系统的安全证书，就可以将程序加载到内核空间以最高权限运行；

当然这也带来了安全隐患，当第三方程序获取了最高权限，可以对硬件设备进行自定义访问时，很容易造成蓝屏、系统损坏等情况；

二、 Linux 内核（Linux Kernel）和操作系统

Linux 精确的说并不是一个操作系统，它是一个操作系统的内核，编写了对硬件的访问、内存管理、基本的文件系统等程序，来规范操作系统对硬件的访问。

不同的厂商根据需求，在 linux 内核的基础上开发出了不同的操作系统，如:centos、ubantu 等，当然操作系统除驱动和内核，其他的部分都在用户空间运行。